Snakes & Ladders AI Solver

Case Study: Finding the Minimum-Throw Solution to Snakes & Ladders Using Uninformed Search (BFS, DFS, IDDFS)

1. Problem formulation

If you can choose your dice rolls, Snakes & Ladders becomes a shortest-path problem on a graph. The game is modelled as a state-space search problem and three uninformed search algorithms are compared.

Element	Definition
State	Current square, 1..100
Initial state	Square 1
Goal state	Square 100 (exact landing; overshooting rolls are illegal)
Actions	Dice roll 1..6
Transition	new = square + roll; if a snake/ladder starts at new, jump to its end
Path cost	1 per throw (minimise throws)

2. Algorithms

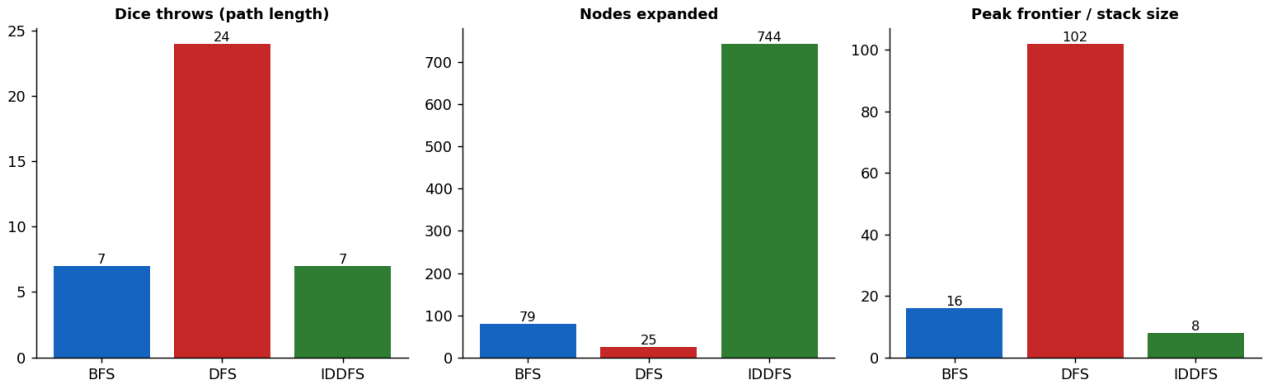
BFS: FIFO queue, level by level. Complete and optimal (unit edge cost).
DFS: LIFO stack with visited set. Returns the first path found, not optimal.
IDDFS: repeated depth-limited DFS. Optimal like BFS with O(depth) memory.

3. Results

Algorithm	Throws	Nodes expanded	Peak frontier	Optimal?
BFS	7	79	16	Yes
DFS	24	25	102	No
IDDFS	7	744	8	Yes

Optimal solution (BFS): 1 -> 14 -> 17 -> 22 -> 84 -> 88 -> 94 -> 100; rolls [3, 3, 5, 6, 4, 6, 6].
Random play over 10,000 games averages 38.4 throws (worst 229); BFS needs 7.

DFS expands the fewest nodes but its path is about 3.4x longer than optimal. BFS guarantees the minimum at the cost of a larger frontier. IDDFS matches BFS with tiny memory but re-expands nodes.



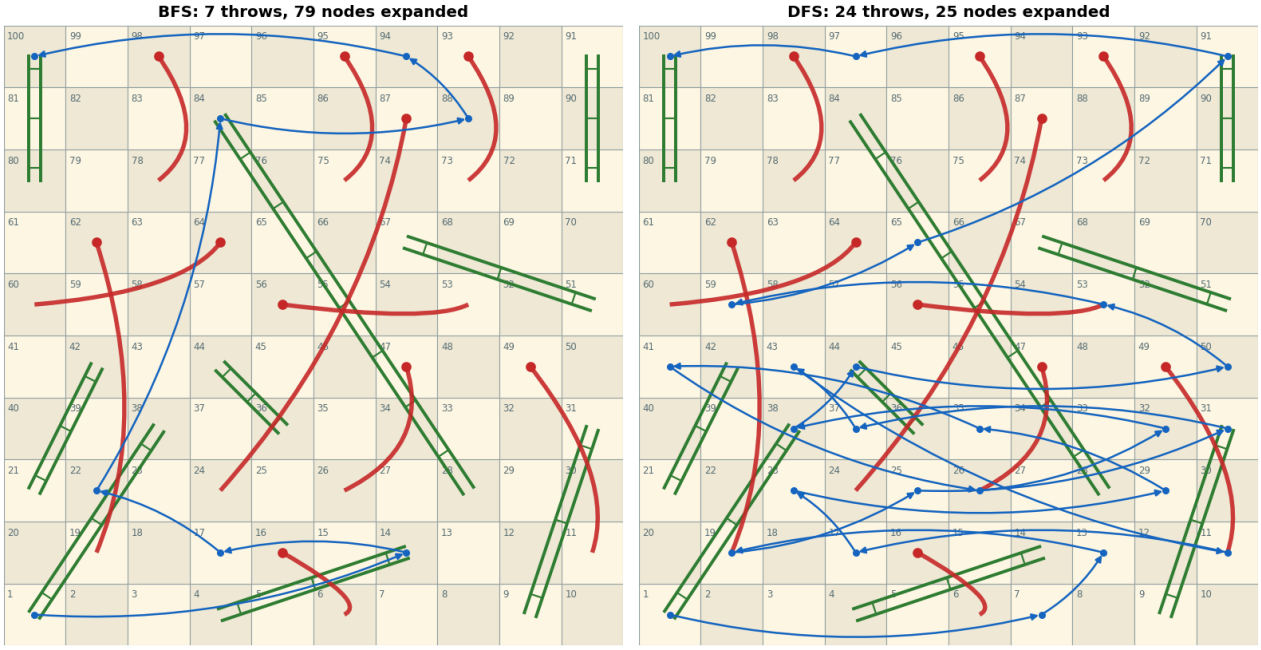


Fig: BFS path (left) vs DFS path (right).

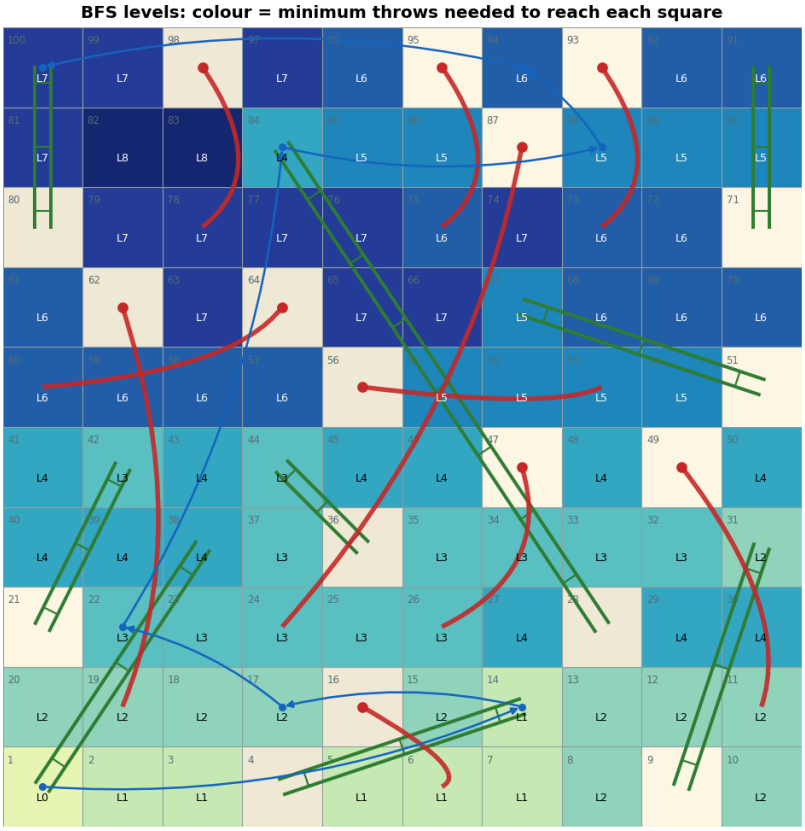
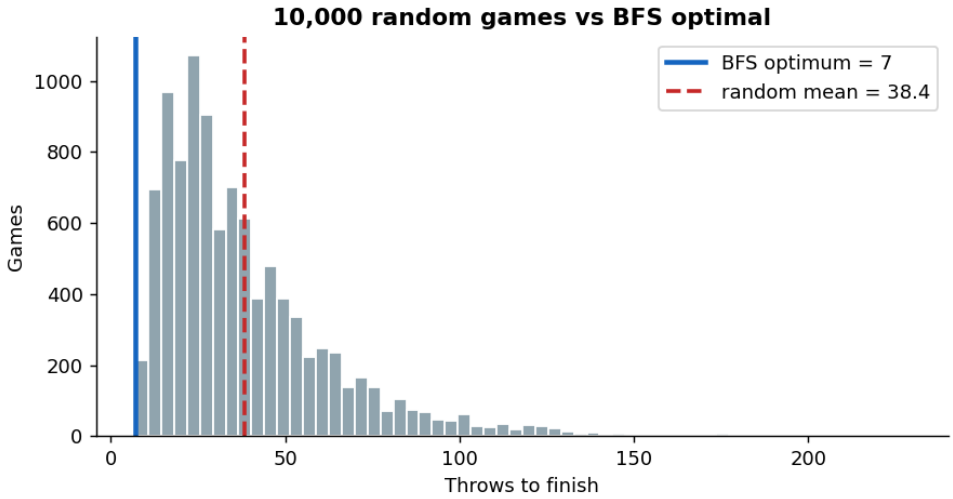


Fig: BFS levels, the minimum throws needed to reach each square.



4. How to run

```
pip install -r requirements.txt
python main.py
python -m unittest -v
```

5. Structure

```
board.py        board model (states, moves, snakes/ladders)
solvers.py      bfs(), dfs(), iddfs()
visualize.py    drawings, charts, animation frames
main.py         runs everything, prints comparison table
test_solvers.py unit tests
output/         figures and demo video
```